



University
of Exeter

COURSEWORK SPECIFICATION

ECM3428 – Algorithms that changed the world

Module Leader: **Chunbo Luo**

Academic Year: 2026/27

Title: **CA1: Algorithm group project**

Submission deadline: **19/11/2026**

This assessment contributes **40%** of the total module mark and assesses the following **intended learning outcomes**:

- Appreciate the principles of algorithm design and implementation;
- Analyse the time complexity of some important classes of algorithms;
- Implement and analyse some fundamental algorithms;
- Apply programming skills to convert abstract specifications into practical realisations;
- Demonstrate an analytical approach to computational problems;
- Appreciate the importance of complexity considerations in the practical deployment of programs at different scales;
- Approach problem-solving tasks in a systematic and disciplined way.

Plagiarism

This is a group assessment. Each group should normally consist of four or five students (maximum 6).

Plagiarism is interpreted by the university as the act of presenting the work of others as one's own work, without acknowledgement. It is considered academically fraudulent and an offence against university discipline. Your attention is drawn to the university's regulations on plagiarism.

Generative AI

This assessment has been categorised as **AI-Assisted**. You can find further information in the University's policies around using AI in assessed work.

You are encouraged to use generative AI tools appropriately, for example to explore ideas, debug code, generate test cases or improve documentation. AI-generated output must not be accepted uncritically. Your group remains responsible for the correctness, security, quality and originality of everything submitted.

All use of generative AI must be acknowledged in the README, including the tool, purpose, representative prompts and how its output was checked. In the interview, every member must be able to explain the

algorithm and submitted implementation without AI assistance. Marks are awarded for demonstrated understanding and the quantity of your creative work.

Instructions

Problem Statement

In this assessment, you will work in a group of four or five students to select one algorithm, implement it and build it into a usable product. The product could be a web application, mobile or desktop application, game, interactive visualisation, developer tool or another suitable software product. It must solve a clearly defined problem and allow a user to see what the algorithm does and why it is useful.

The submitted product must:

- contain a working implementation of one substantial algorithm selected by the group;
- use the algorithm as a key part of the product, rather than as a disconnected separate demonstration;
- allow users to provide, select or modify meaningful inputs and inspect the resulting outputs;
- explain or visualise the important stages of the algorithm sufficiently for a user to understand how the result was produced;
- include appropriate validation, test cases and evidence that the implementation produces correct results and provide clear instructions;
- discuss the algorithm's time and space complexity (this could include consideration of the real-world scenarios) and show awareness of how performance changes with varied settings; and
- identify any important limitations, assumptions, ethical issues or potential improvements.

You may use established libraries, frameworks and APIs to develop the interface and supporting features. However, your group must implement the core algorithmic logic yourselves. A submission that only calls an existing implementation of the selected algorithm as a black box will not demonstrate the required learning outcomes. Where complete implementation from first principles would be unrealistic, you must agree the boundary of your own implementation with the lecturer in advance.

Example Projects

The following are just examples for your inspiration. You may select one of them or propose a different project based on another suitable algorithm. Groups are expected to make their own justified design decisions and add features that create a coherent and useful product. You can use AI or other suitable tools to learn your chosen algorithm.

1. **Smart Route Planner – A* or an improved A* algorithm.** Build an interactive route-planning application in which users create or select a map, choose start and destination points, and add obstacles or costs. Display the explored nodes and final route, report its cost and running time, and compare the selected A* variant with Dijkstra's algorithm or standard A*.
2. **Network Capacity Planner – Edmonds–Karp maximum-flow algorithm.** Design a product that models a transport, communication, water or supply network. Users should be able to create or upload a network, calculate the maximum flow, identify bottlenecks and explore how capacity changes affect the result. Visualise augmenting paths and residual capacities.
3. **Market Basket Insight Tool – FP-Growth algorithm.** Make an application that accepts transaction data (e.g., Amazon products or Apple Store purchases) and discovers frequent itemsets. Users

should be able to change the minimum-support threshold, filter and inspect patterns, and view useful summaries or association rules. Include a small example dataset and explain how data size and support affect performance.

4. **Spatial Search Explorer – k-d tree algorithm.** Build an interactive spatial-data application supporting nearest-neighbour and orthogonal range queries. Users could search locations, facilities, game objects or scientific observations. Visualise how the tree partitions the space and compare query results and performance with a brute-force search.
5. **Multicast Network Designer – multicast routing algorithm.** Develop a network-planning tool that connects one source to multiple destinations while optimising an appropriate measure such as delay, bandwidth use or total path cost. Let users edit the topology and requirements, visualise the resulting multicast structure, and examine failure or congestion scenarios.
6. **Low-Cost Infrastructure Designer – Borůvka's algorithm.** Design a planning application for connecting sites such as homes, sensors, substations or campuses at minimum total cost. Show the components and selected edges at each iteration, calculate the final cost, and compare the result or behaviour with another minimum-spanning-tree algorithm.
7. **Error-Resilient Data Lab – Reed–Solomon algorithm.** Build a product that encodes data, simulates missing or corrupted symbols, and attempts recovery. Clearly show the encoded data, introduced errors and recovered result. Let users vary relevant parameters and explain the number of errors that can and cannot be corrected.
8. **Geospatial Boundary Mapper – Quickhull algorithm.** Make an application in which users enter, generate or upload two-dimensional points and calculate their convex hull. Animate or visualise the important steps, report useful geometric measures, and test the implementation on random, clustered, collinear and large datasets.
9. **Interactive Equation Solver – Jacobi or Gauss–Seidel iterative algorithm.** Build a product that accepts a system of linear equations, checks relevant conditions, shows convergence over successive iterations and compares the numerical result with a reference solution. Allow users to alter the tolerance and iteration limit and explore convergent and non-convergent examples.
10. **Black-Box Optimisation Workbench – a derivative-free optimisation algorithm.** Build an application that lets users optimise a function or practical design without using derivatives. Visualise candidate solutions and progress, expose meaningful parameters, and compare the selected algorithm with random search or another simple baseline on several test problems.

A group wishing to try a different algorithm should discuss the choice with the lecturer to ensure that the algorithm and planned scope are suitable. The proposal should identify the potential algorithms, the problem, the intended users, the proposed product and the main features.

Group Working

Each group should agree roles and responsibilities early. All members must contribute to the algorithmic and product-development work and understand the complete product. Use appropriate project-management and version-control practice and retain evidence of individual contributions. The product receives a group mark.

Interview

Each group will attend a ten-minute interview. The interview will normally consist of:

- **five minutes for the group presentation and live product demonstration;** and
- **five minutes for questions from the assessors.**

The presentation should briefly introduce the problem, explain the chosen algorithm and its role in the product, and demonstrate the most important product features. The demonstration should use prepared test cases, if appropriate, and be self-contained; should not depend on external services unless a reliable fallback is available.

All group members must attend, present and answer questions. Questions may cover the algorithm and its complexity, implementation, testing, limitations, individual contributions and use of generative AI. Assessors may ask you to explain any part of the source code. The interview will be recorded.

Deliverables

The deliverables for this coursework comprise one **electronic group submission** and one **ten-minute group interview**.

- The complete source code and all files required to run the product. Do not include secrets, personal data or unnecessary generated files.
- A working URL for a deployed product or, where deployment is not appropriate, a packaged executable or clear instructions for running the product locally.
- A README containing:
 - the project title, selected algorithm, problem and intended users;
 - installation, access and testing instructions;
 - a concise explanation or pseudocode of the algorithm and its time and space complexity;
 - the test cases used and a summary of the results;
 - known limitations, dependencies and third-party resources;
 - a statement of each member's contributions; and
 - the required acknowledgement of generative AI use.
- The presentation slides in Powerpoint or PDF format.

The source code, README, slides and any other required files should be compressed into one single file and submitted to ELE via the submission link. The submitted README must clearly provide the product URL or local running instructions.

You are suggested to complete the submission at least a few hours before the deadline, leaving enough time for potential last-minute amendments.

Marking criteria

Criteria	Marks
Product ▪ The degree to which the algorithm is meaningfully integrated into a coherent product that addresses a clear problem (15). ▪ The degree to which the selected algorithm is implemented correctly and robustly, rather than used only through a black-box library (15). ▪ The quality of testing and evaluation, including suitable test cases, evidence of correctness, performance or scalability analysis, and comparison with a relevant baseline where appropriate (10). ▪ The usability and clarity of the algorithm explanation or visualisation, the overall originality and professional finish of the submitted product (10).	50
Interview ▪ The clarity, organisation and effectiveness of the group presentation and live product demonstration, including appropriate visual aids and contribution to delivery (15). ▪ The understanding of the algorithm's principles, correctness, time and space complexity, assumptions and limitations (15). ▪ The ability to explain and justify relevant implementation, product-design and testing decisions using evidence from the submitted work (10). ▪ The responses to questions, including reflection on their contribution, challenges, improvements and responsible use of generative AI (10).	50
<i>Total</i>	100

Each student's final mark will be calculated using the WebPA peer assessment tool and be based on the group's mark. Details of the peer assessment tool and calculation is provided in an appendix on the ELE page.